PCET Education Trust

Pimpri Chinchwad College OF Engineering, Pune

Database Management Systems Lab

ASSIGNMENT NO. 08

Name	PRN
Atharva Lokhande	124B1F009

Batch A1

Guided By: Mrs. Alpana Borse Ma'am

Aim:

Aggregation and Indexing: Design and Develop Queries using aggregation and indexing with suitable example using Mongo DB.

Objectives:

- **To implement database indexing techniques** by creating single-field, compound, text, and unique indexes to significantly optimize query performance and enforce data constraints.
- **To design multi-stage aggregation pipelines** using stages like $match, $group, $sort, and $limit to transform raw documents into meaningful statistical insights.
- **To execute complex relational lookups** by utilizing the $lookup and $unwind stages to perform left outer joins between disparate collections, such as linking flights to airlines.
- **To perform advanced data categorization** through the use of $bucket for range-based grouping and $project for dynamic field transformation and calculated schema enhancements.

Case Study: Airline Management System

SQL Queries:

Index Management

Creating indexes improves query performance. This session covers single-field, compound, text, and unique indexes with partial filter expressions.

```
airlines> db.flights.createIndex({ price: 1 }) price_1
airlines> db.flights.createIndex({ source: 1, destination: 1
}) source_1_destination_1
airlines> db.passengers.createIndex(
|    { phone: 1 },
|    {
|      unique: true,
|      partialFilterExpression: { phone: { $exists: true, $type: "string" } }
|    } | ) phone_1 airlines> db.flights.createIndex({
flight_name: "text" }) flight_name_text airlines>
db.flights.dropIndex("price_1")
{ nIndexesWas: 4, ok: 1 }
```

Basic Aggregation and Grouping

Using *$group and $count* to perform mathematical operations like sums, averages, and finding extrema across the dataset.

```
airlines> db.flights.aggregate([
|    { $group: { _id: "$airline_id", count: { $sum: 1 } } }
| ])
[ { _id: 3, count: 1 }, { _id: 1, count: 1 }, { _id: 2, count: 1 }, { _id: 5,
count: 1 } ]


airlines> db.flights.aggregate([
|    { $group: { _id: null, avgPrice: { $avg: "$price" } } }
| ])
[ { _id: null, avgPrice: 5420 } ]
```

```
airlines> db.passengers.aggregate([
|    { $group: { _id: "$gender", avgAge: { $avg: "$age" } } }
| ])
[ { _id: 'F', avgAge: 28 }, { _id: 'M', avgAge: 33.666666666666664 } ]
```

Multi-Stage Pipelines

Combining *$match, $sort, $limit, and $skip* to filter, order, and paginate results effectively within the aggregation pipeline.

```
airlines> db.flights.aggregate([
|    { $match: { price: { $gt: 5000 } } },
|    { $group: { _id: "$source", count: { $sum: 1 } } }
| ])
[ { _id: 'Mumbai', count: 1 }, { _id: 'Delhi', count: 1 } ]

airlines> db.flights.aggregate([
|    { $sort: { price: 1 } },
|    { $limit: 3 }
| ])
[
  { _id: 102, flight_name: 'IN202', source: 'Pune', price: 4500, airline_id: 2 },
{ _id: 105, flight_name: 'AK505', source: 'Bangalore', destination: 'Delhi',
price: 4800, airline_id: 5 },
  { _id: 103, flight_name: 'SJ303', source: 'Delhi', destination: 'Goa', price:
6000, airline_id: 3 }
]
```

Relational Lookups and Unwinding

The *$lookup* stage performs left outer joins with other collections, while *$unwind* deconstructs resulting arrays into individual documents.

```
airlines> db.flights.aggregate([
|    {
|      $lookup: {
|        from: "airlines",
|        localField: "airline_id",
|        foreignField: "_id",
|        as: "airline_info"
|      }
```

```
|    },
|    { $unwind: "$airline_info" },
|    { $project: { flight_name: 1, "airline_info.name": 1 } }
| ])
[
  { _id: 101, flight_name: 'AI101', airline_info: { name: 'Air India' } },
  { _id: 102, flight_name: 'IN202', airline_info: { name: 'Indigo' } },
  { _id: 103, flight_name: 'SJ303', airline_info: { name: 'SpiceJet' } },  { _id:
105, flight_name: 'AK505', airline_info: { name: 'Akasa Air' } }
]
```

Advanced Data Transformation

Using *$project* for field inclusion and calculation, *$bucket* for categorizing data into ranges, and *$addFields* for dynamic schema enhancement.

```
airlines> db.passengers.aggregate([
|    {
|      $bucket: {
|        groupBy: "$age",
|        boundaries: [ 0, 18, 30, 50, 100 ],
|        default: "Other"
|      }
|    }
| ])
[ { _id: 18, count: 2 }, { _id: 30, count: 2 } ]

airlines> db.flights.aggregate([
|    { $addFields: { tax: { $multiply: [ "$price", 0.05 ] } } }
| ])
[
  { _id: 101, flight_name: 'AI101', price: 6380.000000000001, tax:
319.00000000000006 },
  { _id: 102, flight_name: 'IN202', price: 4500, tax: 225 }
]
```

Case Study: Civic Voice Interface System SQL Queries:

Index Configuration

Indexes are established to optimize query performance and enforce data integrity, such as unique email constraints and multi-field geographic searches.

```
civic_voice> db.user.createIndex({ email: 1 }, { unique: true })
email_1
civic_voice> db.complaint.createIndex({ status: 1 }) status_1
civic_voice> db.citizenprofile.createIndex({ state: 1, district: 1
}) state_1_district_1
civic_voice> db.complaint.createIndex({ title: "text" })
title_text
civic_voice> db.complaint.dropIndex("status_1")
{ nIndexesWas: 3, ok: 1 }
```

Basic Aggregation and Statistics

Using the *$group and $count* stages to summarize user roles, complaint statuses, and demographic statistics like average and maximum ages.

```
civic_voice> db.user.aggregate([
|    { $group: { _id: "$role", count: { $sum: 1 } } }
| ])
[ { _id: 'Moderator', count: 1 }, { _id: 'Citizen', count: 2 } ]


civic_voice> db.citizenprofile.aggregate([
|    { $group: { _id: "$state", avgAge: { $avg: "$age" } } }
| ])
[ { _id: 'Karnataka', avgAge: 29 }, { _id: 'Maharashtra', avgAge: 31 } ]
```

Filtering, Sorting, and Pagination

Combining *$match with $sort, $skip, and $limit* allows for precise data retrieval and the implementation of paginated views.

```
civic_voice> db.citizenprofile.aggregate([
|    { $sort: { age: -1 } }
| ])
[
  { _id: 103, user_id: 3, state: 'Maharashtra', age: 33, city: 'Mumbai' },
  { _id: 101, user_id: 1, state: 'Maharashtra', district: 'Pune', age: 29 },
  { _id: 104, user_id: 4, state: 'Karnataka', district: 'Bangalore', age: 29 }
]


civic_voice> db.user.aggregate([
|    { $skip: 1 },
|    { $limit: 2 }
| ])
[
  { _id: 2, name: 'Neha Verma', email: 'neha@email.com', role: 'Citizen', is_active:
true },
  { _id: 3, name: 'Amit Patil', email: 'amit@email.com', role: 'Citizen', is_active:
true }
]
```

Relational Joins and Data Shaping

The *$lookup* stage performs joins between collections, which are then flattened using *$unwind* and refined using *$project* to create a specific output schema.

```
civic_voice> db.complaint.aggregate([
|    {
|      $lookup: {
|        from: "user",
|        localField: "user_id",
|        foreignField: "_id",
|        as: "u"
|      }
|    },
|    { $unwind: "$u" },
|    { $project: { title: 1, status: 1, "u.name": 1, "u.email": 1 } } | ])
[
```

```
  { _id: 1, title: 'Pothole on Road', status: 'Pending', u: { name: 'Rahul Sharma',
email: 'rahul@email.com' } }
]
```

Advanced Analysis and Transformation

Utilizing *$bucket* for age categorization, *$addFields* for conditional logic, and *$addToSet* to collect unique values within groups.

```
civic_voice> db.citizenprofile.aggregate([
|    {
|      $bucket: {
|        groupBy: "$age",
|        boundaries: [ 18, 30, 45, 60, 100 ],
|        default: "Unknown"
|      }
|    }
| ])
[ { _id: 18, count: 2 }, { _id: 30, count: 1 } ]

civic_voice> db.citizenprofile.aggregate([
|    { $group: { _id: "$state", districts: { $addToSet: "$district" } } }
| ])
[
  { _id: 'Maharashtra', districts: [ 'Pune' ] },
  { _id: 'Karnataka', districts: [ 'Bangalore' ] }
]

civic_voice> db.complaint.aggregate([
|    {
|      $lookup: {
|        from: "citizenprofile",
|        localField: "user_id",
|        foreignField: "user_id",
|        as: "profile"
|      }
|    },
|    { $unwind: "$profile" },
|    { $group: { _id: "$profile.state", total_complaints: { $sum: 1 } } }
| ])
[ { _id: 'Maharashtra', total_complaints: 1 } ]
```

Conclusion:

This assignment successfully demonstrated the power of **Aggregation and Indexing** in MongoDB. By implementing various indexing strategies-such as compound indexes for geographic searches and unique indexes for user security-I ensured that the database remains performant even as the volume of data grows.

The use of the **Aggregation Framework** allowed for deep data analysis that goes far beyond simple CRUD operations. Stages like $group and $bucket enabled the calculation of complex metrics like average flight prices and citizen age demographics, while $lookup bridged the gap between collections to provide a holistic view of the system. These tools are essential for building data-driven features in both the **Airline Management System** and the **Civic Voice Interface**, providing the analytical capabilities required for modern application backends.